



The "Big 5" Infosys Pattern Combinations

For SP (Specialist Programmer) and DSE (Digital Specialist Engineer) roles, the 3 questions generally map to these specific combinations.

1. Prefix Sum + Hash Map (The "Subarray" Combo)

Frequency: Very High (Usually the "Easy" or "Medium" question)

- **The Story:** "A financial analyst is looking at a log of daily profits and losses. Find the longest consecutive period where the net profit was exactly K (or zero)."
- **The Trap:** A brute-force approach (nested loops) will give you a Time Limit Exceeded (TLE) error. Infosys expects $O(N)$ time.
- **The Mechanism:** As you iterate through the array, maintain a `running_sum`. Store every `running_sum` and its `index` in a Hash Map. At every step, check if `(running_sum - K)` already exists in your map. If it does, boom—you just found your subarray.
- **Practice These:**
 - Subarray Sum Equals K
 - Longest Subarray with 0 Sum
 - Count Subarrays with sum divisible by K

2. 2D Matrix + BFS/DFS (The "Grid" Combo)

Frequency: High (Often the "Medium" or "Hard" question)

- **The Story:** "An autonomous robot needs to navigate a warehouse grid from top-left to bottom-right. Some cells contain obstacles (1s), and open paths are (0s). Find the shortest path, or count the number of isolated storage zones."
- **The Trap:** People try to use complex Graph algorithms (like Dijkstra's) when a simple Breadth-First Search (BFS) is enough.
- **The Mechanism:** Treat the 2D array as a graph. Use a Queue for BFS (best for "shortest path") or recursion for DFS (best for "counting areas"). Always use a `visited[][]` boolean matrix and a direction array `int[][] dirs = {{0,1}, {1,0}, {0,-1}, {-1,0}}` to move Up, Down, Left, Right.
- **Practice These:**
 - Number of Islands
 - Rotting Oranges
 - Shortest Path in Binary Matrix

3. Sliding Window + Two Pointers (The "Sequence" Combo)

Frequency: High (Usually the "Easy" question)

- **The Story:** "A network router receives a stream of data packets represented by a string. Find the longest sequence of packets without any repeating packet IDs."

- **The Trap:** Restarting the search from scratch every time you find a duplicate.
- **The Mechanism:** Use a `left` and `right` pointer to create a "window". Expand the `right` pointer to include new elements. If a rule breaks (e.g., you find a duplicate), don't start over; just shrink the `left` pointer until the window is valid again.
- **Practice These:**
 - Longest Substring Without Repeating Characters
 - Max Consecutive Ones III
 - Minimum Window Substring (Harder variation)

4. Greedy + Priority Queue / Min-Heap (The "Optimizer" Combo)

Frequency: Medium/High (Alternative to standard sorting Greedy)

- **The Story:** "You manage a CPU that must process various tasks. Each task has a cost to merge with another task. Connect all cables/tasks with the minimum possible cost."
- **The Trap:** Sorting the array once isn't enough, because combining items creates *new* values that also need to be sorted dynamically.
- **The Mechanism:** Throw all elements into a Priority Queue (Min-Heap). Pop the two smallest elements, process them, and push the result *back* into the Queue. The Queue automatically keeps everything sorted for you in $O(\log N)$ time.
- **Practice These:**
 - Minimum Cost to Connect Ropes
 - Meeting Rooms II (Tracking active rooms)
 - Task Scheduler

5. Dynamic Programming + Strings (The "Comparison" Combo)

Frequency: High (The classic "Hard" question)

- **The Story:** "A geneticist is comparing two DNA strands. What is the minimum number of mutations (insertions, deletions, substitutions) required to turn Strand A into Strand B?"
- **The Trap:** Trying to use greedy logic or two-pointers. Strings that require tracking multiple overlapping possibilities *a/ways* require DP.
- **The Mechanism:** Create a 2D array `dp[i][j]`. Compare character `i` of String 1 and character `j` of String 2. If they match, `dp[i][j] = dp[i-1][j-1]`. If they don't, take the minimum/maximum of the surrounding cells (Insert, Delete, Replace).
- **Practice These:**
 - Longest Common Subsequence (LCS)
 - Edit Distance
 - Longest Palindromic Subsequence